

Chris Young

Logan Douglas

Nathan Howell

SLO1 - Complex Engineering Problem

Introduction

Our initial idea for the project was something that a user could use to get buy and sell recommendations for a given stock. While we knew we wanted to have something related to the stock market and providing value with potentially giving users an edge in making trades. We would then later refine this until the project that our group decided on was to develop a stock screening application that would try to provide a way to predict the price of a stock a few hours into the future.

This would then be expanded further to have the intent of trying to help inform users about information relating to the stock market putting more emphasis on what different filters and features did. We wanted to try and give users who were new to the stock market a starting point.

Explanation of Problem

One of the biggest challenges while working through this project was the problem of gathering, managing, and updating stock data to display to users. Creating a stock screener involves a lot of data from the market that we initially had no idea how we were going to get meaningfully. In order to display all of this data our team had to come up with a solution to handle large quantities of data. It was also important to make sure we were getting fresh data to supply to our application.

While trying to gather the data required we began to consider what we needed, creating a list of requirements for what was needed in a long term solution. There were also some constraints to consider. For us we needed this to be cost effective, work with our existing software, and be able to run on the class servers. With this in mind we created this list with three main goals in mind; Able to handle a large amount of data throughput, at least 2 years worth of financial data, possible to get all stock data within the last hour. To meet these goals we thought about a few different methods.

Working the Problem

There were many different approaches we thought about such as scraping the data ourselves from different sources, using an established API in order to get stock data, or paying a third party for the data we were looking for. There was also the management of this data which brought us to a couple of solutions, managing everything on a single service, or splitting it into many different ones.

The first was pulling data from many different sources and combining it together for our needs. By pooling the data from different sources we had the ability to get at least 2 years worth of data, could get stock data within the last hour, and on the surface appeared to allow us to handle large amounts of data. This approach would give us everything we were looking for without costing us much money. We didn't know until we implemented this approach that this would be the wrong way to handle it. After implementation one of the core services used to get this stock information, yfinance, had very serious rate limiting that a project at our scale quickly exceeded.

After prototyping and experimenting with this we decided a different approach was necessary. Our second attempt had us looking at one source of data, Polygon, and paying for the data that we needed in order to gather what we were looking for. As we experimented with this solution it was getting closer to our desired results. It allowed us to handle large quantities of data without rate limits, had extensive market history for every stock, and could get us up to the last minute's worth of finance data for roughly 12,000 stocks. We believed we could now handle large amounts of data so we went with this approach and created prototypes.

The next challenge we faced was how we were hosting and managing this large amount of data. We initially went with hosting a small database and managing the data in a single service. While we had a good source of data, we found our application was slow due the way we were collecting that data in our database and where we were hosting our database. With these issues we went back to come up with a new design

The final design of our stock data fetcher involved creating an orchestrator service that would manage a group of services. These services would then make calls to Polygon's API every hour in order to populate a database hosted on Azure. This orchestrator would also manage a calculation service which would work with the data gathered from each call in order to calculate values such as Earnings Per Share and Outstanding Shares. With all of these services having the goal of updating a batch of stocks given to them it allowed for everything to be updated quickly creating an accurate and fast experience for users using our application.

Solution to the Problem

We proved that our solution works by creating prototypes of the approaches we took to see if there were any flaws in the approach we wanted to take. For our final solution we created these services and had them populate a database for two years worth of data to see how fast and accurate the data we received was. Comparing the speed of our initial approach to what we got used as our final solution we were able to get our data inside of a database almost 20 times faster.

We then deployed the implementation to have users test the interactivity of the application noting down times when things were slower than they were expecting or where it had sped up compared to previous implementations. We also monitored the time it took through backend logs and traces to help make sure the results we were getting were sufficient. Our solution was cheap and worked with the systems we already had in place. When we were able to gather and handle large amounts of data, get two years worth of financial history, and were getting data for each hour; we knew we had come to the solution we were looking for. This solution also met our constraints. It worked on our class servers, was cost effective, and worked on the existing software we were using.

Final Reflection

Afterwards as a team we learned to place a stronger importance on coming up with more potential options before deciding what to go with. This was something our group didn't consider while coming up with designs. We learned that by putting more

value into creating options and weighing them it could have avoided some headaches we faced.

We also learned a lot about making prototypes and experimenting with different solutions. There were many times that as a group we had to create an example of something before we could bring it over to our app thanks to this problem that we had. Such as creating the first microservice to fetch stock data. There was a lot more nuance than we expected so having something prototyped out made it much easier to come to an informed conclusion about if our group was heading in the right direction for the project.

If there were a constraint that could be changed such as the budget we could spend there was the possibility to get data streamed directly from one source to us that we could use in our application. By spending more money we also could have gained better insight into company's financial reports to get more data for our users and model. I believe the biggest difference with this would be relying less on hourly data and our hand calculations in order to provide a more accurate solution to consumers.